# Solution: Problem Challenge 2: Comparing Strings containing Backspaces

## Problem Statement

Given two strings containing backspaces (identified by the character '#'), check if the two strings are equal.

**Example 1:**

```
Input: str1="xy#z", str2="xzz#"
Output: true
Explanation: After applying backspaces the strings become "xz" and "xz" respectively.
```

**Example 2:**

```
Input: str1="xy#z", str2="xyz#"
Output: false
Explanation: After applying backspaces the strings become "xz" and "xy" respectively.
```

**Example 3:**

```
Input: str1="xp#", str2="xyz##"
Output: true
Explanation: After applying backspaces the strings become "x" and "x" respectively.
In "xyz##", the first '#' removes the character 'z' and the second '#' removes the character 'y'.
```

**Example 4:**

```
Input: str1="xywrrmp", str2="xywrrmu#p"
Output: true
Explanation: After applying backspaces the strings become "xywrrmp" and "xywrrmp" respectively.
```

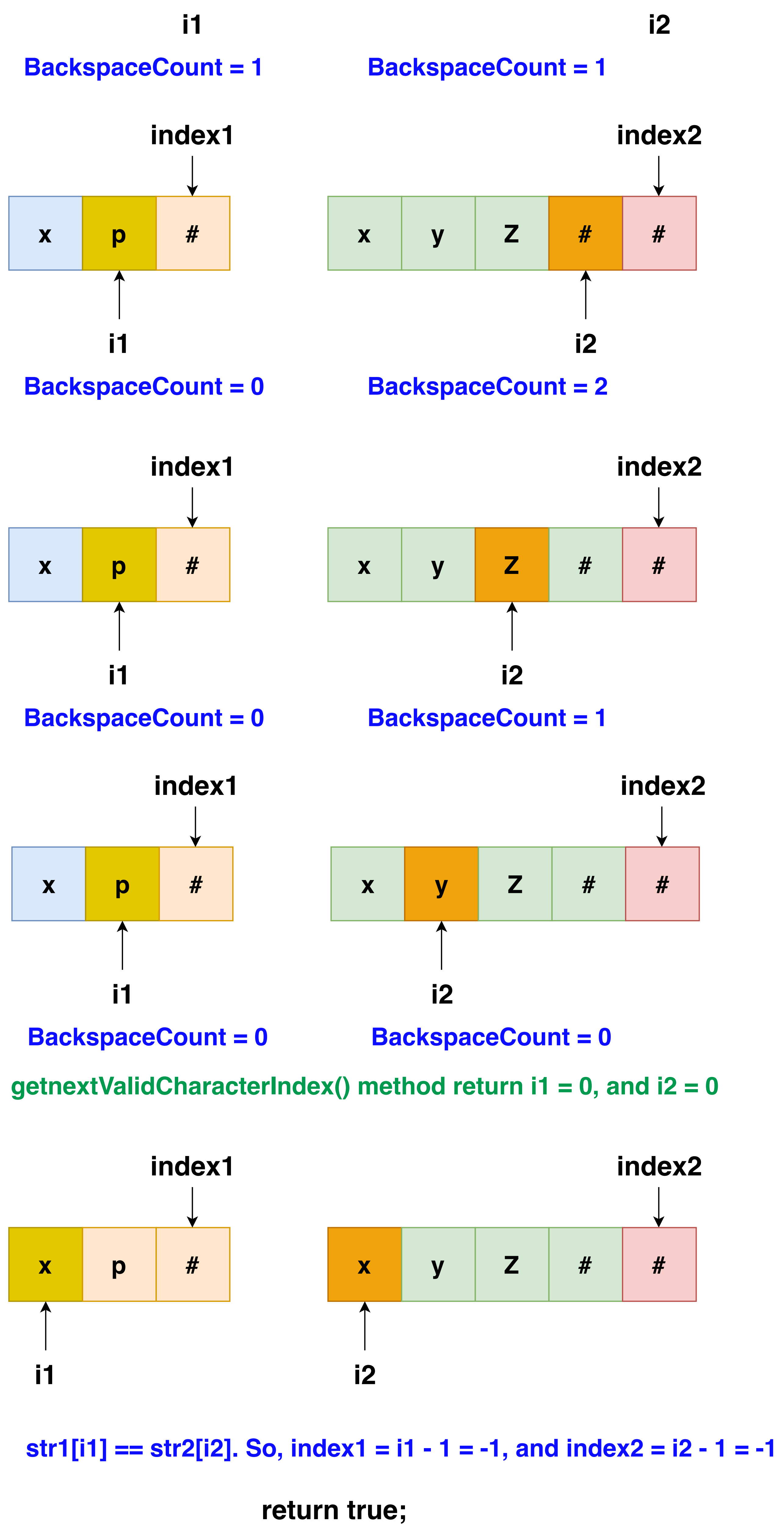
## Solution

To compare the given strings, first, we need to apply the backspaces. An efficient way to do this would be from the end of both the strings. We can have separate pointers, pointing to the last element of the given strings. We can start comparing the characters pointed out by both the pointers to see if the strings are equal. If, at any stage, the character pointed out by any of the pointers is a backspace ('#'), we will skip and apply the backspace until we have a valid character available for comparison.

Here's a detailed walkthrough of the algorithm:

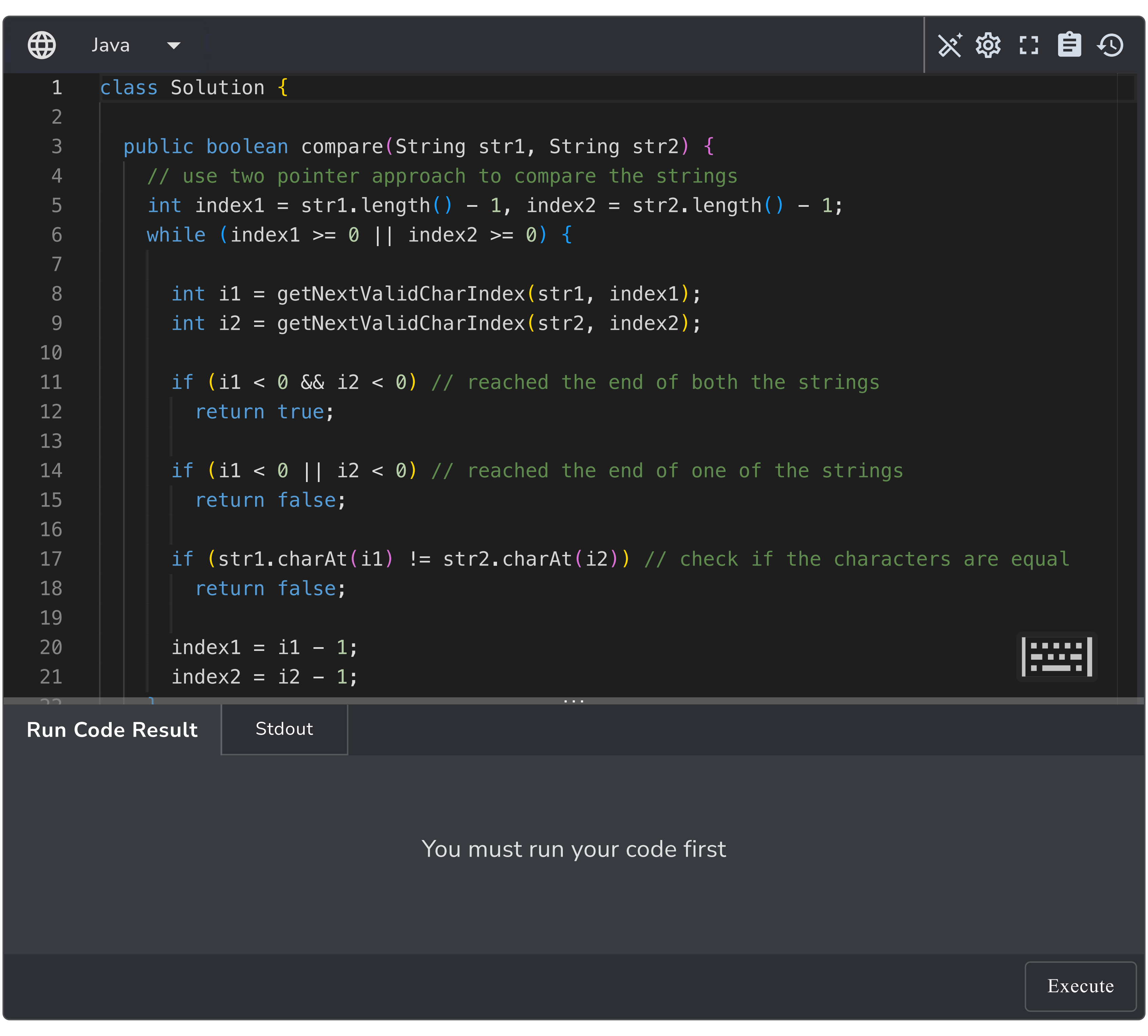
1. In the `compare` function, two pointers are initialized, `index1` and `index2`, to the last index of `str1` and `str2`, respectively.
2. A while loop is started which continues until both pointers are less than zero, that is, both have traversed their strings completely in a reverse manner.
3. Inside this loop, for each string, the `getNextValidCharIndex` function is called with the current pointer. This function returns the index of the next valid character in the string (traversing from back to front) by taking into account the backspace characters. `i1` and `i2` point to the index of the next valid character in the two strings.
4. If both `i1` and `i2` are less than zero, it means the end of both strings has been reached, and the strings are considered equal.
5. If only one of `i1` or `i2` is less than zero, it means the end of one string has been reached, but not the other, and the strings are not equal.
6. If the characters at indices `i1` and `i2` are not the same, the strings are not equal.
7. If none of the above conditions are met, the loop continues to the next valid characters in both strings.
8. The `getNextValidCharIndex` function accepts a string and an index, and uses a backspace count to keep track of how many backspaces have been encountered. It then loops through the string backwards from the provided index until it encounters a valid character or reaches the beginning of the string.
9. If a backspace character is found, the backspace count is incremented. If a non-backspace character is found and there are any counted backspaces, one backspace is subtracted from the count (to simulate the removal of the previous character), and the loop continues. If a non-backspace character is found and there are no backspaces left to account for, the loop breaks and the index of the valid character is returned.

Here is the visual representation of this algorithm for Example 2:


input: str1 = "xp#", str2 = "xy##"
getnextValidCharacterIndex() method is executed
index1
index2
x
p
#
x
y
Z
#
#
i1
i2
BackspaceCount = 1
BackspaceCount = 1
BackspaceCount = 0
BackspaceCount = 2
BackspaceCount = 0
BackspaceCount = 1
BackspaceCount = 0
BackspaceCount = 0
getnextValidCharacterIndex() method return i1 = 0, and i2 = 0
str1[i1] == str2[i2]. So, index1 = i1 - 1 = -1, and index2 = i2 - 1 = -1
return true;

## Code

Here is what our algorithm will look like:



```java
class Solution {

  public boolean compare(String str1, String str2) {
    // use two pointer approach to compare the strings
    int index1 = str1.length() - 1, index2 = str2.length() - 1;
    while (index1 >= 0 || index2 >= 0) {

      int i1 = getNextValidCharIndex(str1, index1);
      int i2 = getNextValidCharIndex(str2, index2);

      if (i1 < 0 && i2 < 0) // reached the end of both the strings
        return true;

      if (i1 < 0 || i2 < 0) // reached the end of one of the strings
        return false;

      if (str1.charAt(i1) != str2.charAt(i2)) // check if the characters are equal
        return false;

      index1 = i1 - 1;
      index2 = i2 - 1;
```

## Time Complexity

The time complexity of the above algorithm will be $O(M+N)$ where 'M' and 'N' are the lengths of the two input strings respectively.

## Space Complexity

The algorithm runs in constant space $O(1)$.

← Previous
Problem Challenge 2: Comparing Strings Containing Backspaces (Medium)

Next →
Problem Challenge 3: Minimum Window Sort (Medium)

☐ Mark as Completed